

PRUEBA TÉCNICA

Aprendiz / Practicante · Frontend Developer

Proyecto: UI Component Library — Angular + TypeScript

El objetivo de esta prueba es diseñar y construir una librería de componentes Angular propia, publicable y reutilizable, y una aplicación demo que la consuma. Se evaluará la capacidad para escribir componentes limpios, bien tipados y correctamente desacoplados, aplicando las convenciones modernas del ecosistema Angular.

Tecnologías: Angular 17+ · TypeScript (strict) · Tailwind CSS v3 · Angular Library · Rick and Morty API

1. Estructura del Proyecto y Configuración

El proyecto debe ser un **Angular workspace** con dos proyectos internos diferenciados. Se evaluará no solo la funcionalidad final, sino también la estructura y calidad del código.

```
my-workspace/
├── projects/
│   ├── ui-lib/ ← La librería de componentes
│   └── demo-app/ ← App Angular que consume la librería
├── angular.json
└── package.json
```

Convenciones obligatorias

- **Standalone + OnPush** — Todos los componentes deben ser standalone con `ChangeDetectionStrategy.OnPush`.
- **Signals API** — Usar `input()`, `output()` y `model()` de Angular 17+. Prohibido usar los decoradores `@Input()` / `@Output()`.
- **Prefix propio** — Definir un prefijo consistente para todos los selectores (ej. `ui-`, `my-`) y aplicarlo en cada componente.
- **Public API limpia** — Exportar todos los componentes desde `public-api.ts`. La demo app no debe hacer importaciones internas directas.
- **Tipado estricto** — `strict: true` en `tsconfig.json`. Cero uso de `any`.

2. Librería de Componentes (ui-lib)

La librería debe contener cuatro componentes. Los estilos se implementan con **Tailwind CSS v3**. No se permite el uso de librerías de componentes externas (Angular Material, PrimeNG, NgBootstrap, etc.).

Componente 1 — Button <ui-button>

Tipo	Nombre	Tipo TS	Descripción
input()	label	string	Texto visible del botón
input()	variant	'primary' 'secondary' 'danger'	Estilo visual del botón
input()	size	'sm' 'md' 'lg'	Tamaño del botón
input()	disabled	boolean	Bloquea la interacción
input()	loading	boolean	Muestra spinner y bloquea el click
output())	clicked	void	Emite solo si no está disabled ni loading

Cada variant debe tener estilos visualmente distintos. Cuando loading es true, mostrar un spinner inline y deshabilitar el evento click.

Componente 2 — Card <ui-card>

Tipo	Nombre	Tipo TS	Descripción
input()	title	string	Título del header de la card
input()	subtitle	string null	Subtítulo opcional bajo el título
input()	elevation	'flat' 'raised' 'outlined'	Estilo visual del contenedor
output())	headerClicked	void	Emite al hacer clic en el header
ng-content	—	—	Proyecta contenido arbitrario en el body

Componente 3 — Select <ui-select>

Tipo	Nombre	Tipo TS	Descripción
input()	options	SelectOption[]	Lista de opciones { label, value }
input()	label	string	Etiqueta visible sobre el select
input()	placeholder	string	Texto cuando no hay selección
input()	loading	boolean	Estado de carga (skeleton o spinner)
input()	disabled	boolean	Bloquea la interacción
model()	value	string null	Two-way binding del valor seleccionado

output())	selectionChange	SelectOption	Emite el objeto completo al cambiar
---------------	-----------------	--------------	-------------------------------------

Componente 4 — Table <ui-table>

Tipo	Nombre	Tipo TS	Descripción
input()	columns	TableColumn[]	Definición de columnas { key, header }
input()	rows	T[] (genérico)	Datos a renderizar
input()	loading	boolean	Muestra skeleton rows en lugar de datos
input()	emptyMessage	string	Mensaje cuando rows está vacío
input()	errorMessage	string null	Mensaje de error de red visible en la tabla
output())	actionTriggered	TableAction	Emite { action: 'view' 'delete', row: T }

La tabla es genérica — no conoce el dominio de los datos. Solo renderiza columnas y emite acciones. El componente padre es responsable de interpretar cada acción recibida.

3. Flujo Principal — Explorer de Recursos (demo-app)

La demo app debe consumir la **Rick and Morty API** (<https://rickandmortyapi.com/api>) e integrar los cuatro componentes de la librería en un flujo cohesivo.

ui-select (Recurso)	Cambia el recurso activo: Characters Episodes Locations. Al cambiar, hace una petición al endpoint correspondiente y alimenta la tabla con los resultados.
ui-select (Filtro)	Filtra por status dentro del recurso activo. Las opciones son fijas: Alive Dead Unknown. El select de filtro se reinicia al cambiar de recurso.
ui-table	Renderiza los registros del recurso activo. El aprendiz decide qué columnas mostrar por recurso. Maneja tres estados: loading (skeleton rows), empty y error.
Ver detalle	Abre un modal con ui-card mostrando todos los campos del registro. El contenido se proyecta vía ng-content según el tipo de recurso. Incluye botón de cerrar.
Eliminar (opcional)	No es obligatorio implementar la eliminación real del estado local. Sin embargo, el botón de eliminar debe existir en la tabla y el output actionTriggered debe emitir correctamente la acción 'delete' con el registro al pulsarlo.

Comportamiento esperado

- Al cargar la app, el select de recurso tiene **Characters** preseleccionado y la tabla muestra los primeros resultados.
- Al cambiar el recurso, el select de filtro se **resetea** a su estado inicial.
- Al cambiar el filtro de status, la tabla hace una nueva petición con el parámetro aplicado.

- Durante cualquier petición, la tabla muestra el **skeleton loading**.
- Si la petición falla, la tabla muestra el **estado de error** con un mensaje claro.
- Al hacer clic en **Ver detalle**, se abre el modal con ui-card mostrando todos los campos del registro.
- Al hacer clic en **Eliminar**, el output actionTriggered emite { action: 'delete', row } correctamente. La eliminación real del estado es opcional.

Importante: el estado del flujo (recurso activo, filtro, registros, loading, error) debe manejarse en un servicio con signals. Los componentes no hacen llamadas HTTP directas.

4. Diseño Visual y UX

Los estilos se implementan con **Tailwind CSS v3** — no instalar v4. El diseño visual de la demo app debe estar **inspirado en el universo de Rick & Morty**: paleta de colores, tipografía y atmósfera deben evocar la serie. No es obligatorio usar assets o imágenes con derechos — se evalúa el criterio y coherencia visual del candidato.

- **Responsivo** — La demo app debe verse correctamente en mobile y desktop.
 - **Estados visuales** — Cada componente debe tener estilos claros para: default, hover, focus, disabled, loading y error.
 - **Feedback al usuario** — Acciones como eliminar deben tener confirmación. El cambio de recurso debe ser fluido visualmente.
 - **Consistencia** — La paleta y espaciados deben ser coherentes en toda la app.
 - **Empty state** — Cuando no hay resultados, mostrar un estado vacío con mensaje ilustrativo, no una tabla en blanco.
-

5. Documentación y Buenas Prácticas

README.md

- Descripción del proyecto y su arquitectura.
- Instrucciones de instalación (npm install) y ejecución (ng serve demo-app).
- Tabla de API por componente: inputs, outputs, modelos y ejemplos de uso en HTML.
- Decisiones de diseño relevantes tomadas durante el desarrollo.

JSDoc y Código

- Documentar con JSDoc todos los inputs, outputs y métodos públicos de los componentes.
- El código debe ser legible y seguir las mejores prácticas de TypeScript.
- Prohibido el uso de any. Definir interfaces claras para Character, Episode y Location.

Conventional Commits

Realizar commits atómicos y descriptivos. Ejemplos:

```
feat: add ui-button component with variant support
```

```
feat: implement ui-table with skeleton loading state
fix: reset filter select on resource change
refactor: extract resource state to signals service
docs: add component API table to README
```

6. Entregables

Código Fuente

Repositorio en GitHub con el workspace completo. El README.md debe incluir toda la documentación necesaria para instalar y ejecutar el proyecto sin asistencia.

Bitácora de Desarrollo — DEVELOPMENT_LOG.md

Este archivo documenta el proceso de desarrollo. Documentar brevemente las decisiones tomadas, los retos encontrados y cómo se resolvieron.

Uso de herramientas de IA

El uso de herramientas de IA (ChatGPT, GitHub Copilot, Claude, etc.) está **permitido**. No es necesario ocultarlo — se valora la transparencia. Sin embargo, durante la revisión de la prueba se harán **preguntas puntuales sobre el código entregado**: cómo funciona, por qué se tomó cada decisión, qué hace cada parte. Lo más importante es que el candidato pueda presentar la prueba con consciencia de lo que se hizo, independientemente de cómo se llegó al resultado.

Para cada uso de IA, documentar:

- **El prompt utilizado** — Cómo se formuló la solicitud.
- **Qué se aceptó y por qué** — Qué partes se incorporaron y qué las hace correctas para este proyecto.
- **Qué se rechazó o modificó** — Qué no se aceptó directamente y por qué (tipado incorrecto, lógica equivocada, etc.).

Un candidato que usa IA con criterio —entendiendo lo que produce— es más valioso que uno que escribe todo sin asistencia sin comprenderlo.

Retos y Soluciones

Describir brevemente los bloqueos o errores encontrados durante el desarrollo y cómo se superaron.

Video explicativo

Un video recorriendo la demo app en funcionamiento y explicando las decisiones de implementación más relevantes de la librería.

7. Deseables (No obligatorios)

- Pruebas unitarias de al menos un componente con **Jest** o **Vitest**.

- Storybook para visualizar los componentes de la librería de forma aislada.
- Publicación del paquete en **GitHub Packages** o npm.
- Despliegue de la demo app en Vercel, Netlify o GitHub Pages.

8. Criterios de Evaluación

#	Criterio	Descripción
1	Estructura del workspace	Separación correcta librería / demo app, public-api.ts limpio, convenciones aplicadas.
2	Convenciones Angular 17+	Uso correcto de input(), output(), model(), OnPush y standalone.
3	Calidad visual y UX	Tailwind aplicado con criterio, tema Rick & Morty coherente, responsivo, estados contemplados.
4	Flujo e integración	El select cambia recurso, el filtro de status funciona, la tabla reacciona y el modal muestra el detalle. El output de eliminar emite correctamente.
5	Tipado y calidad del código	Interfaces definidas para cada recurso, tabla genérica, cero any, código limpio.
6	Documentación y Git	README con tabla de API, JSDoc en componentes, commits atómicos y descriptivos.
7	Consciencia del código entregado	Se harán preguntas puntuales sobre el código. Lo más importante es que el candidato pueda explicar y defender lo que entrega, independientemente de las herramientas usadas.

9. Recursos de Apoyo

Estos recursos pueden ayudarte a familiarizarte con las tecnologías usadas en la prueba. No son un tutorial paso a paso — se espera criterio propio al aplicarlos.

Angular — Librería y Workspace

- Documentación oficial — angular.dev/tools/libraries/creating-libraries
- Guía práctica librería + demo app — dev.to/yljerjen/angular-library-with-demo-project-1i1b
- Guía completa de Angular libraries — willtaylor.blog/complete-guide-to-angular-libraries
- Referencia CLI ng generate library — angular.dev/cli/generate/library

Angular — Signals API

- Documentación oficial signals — angular.dev/guide/signals
- Guía completa input(), output(), model() — blog.angular-university.io/angular-signal-components
- Signal inputs en detalle — blog.angular-university.io/angular-signal-inputs

Tailwind CSS v3

- Instalación con Angular (v3) — v3.tailwindcss.com/docs/guides/angular
- Documentación general v3 — v3.tailwindcss.com/docs/installation

Rick and Morty API

- Documentación oficial y endpoints — rickandmortyapi.com/documentation

Plazo de Entrega: 4 días calendario desde la recepción de este documento.

Instrucciones finales: Enviar el enlace al repositorio de GitHub y el video explicativo.

Contacto para dudas: Jonathan Martinez — WhatsApp 322 474 7761